

CHICK BLAST

Next-Generation Quick Service Restaurant (QSR) Web Platform, Real-Time Kitchen Display System (KDS) & POS Operations Engine

🌐 Live Deployment: chick-blast.vercel.app

📄 Version: 1.1.2

⚡ Architecture: Full-Stack SPA & REST API

📅 Updated: September 2026

📌 1. Executive Summary & Overview

Chick Blast is an end-to-end commercial restaurant technology platform designed for fried chicken outlets, cloud kitchens, and takeaway diners. The ecosystem delivers a frictionless omni-channel customer ordering experience coupled with an automated back-of-house Kitchen Display System (KDS) and store management analytics.

**Customer Storefront**
High-converting, responsive food catalog with real-time search, interactive hero carousel, instant cart calculation, and multi-mode fulfillment (Dine-In, Takeaway, Home Delivery).

**Kitchen Display (KDS)**
Paperless kitchen operation pipeline with live order statuses, preparation countdowns, customer order tickets, and one-tap stage progression.

**POS & Analytics**
Store management dashboard with revenue analytics, Recharts visualizations, dynamic product CRUD, bundle/combo deals builder, and date-filtered order reporting.

🔧 2. Comprehensive Technology Stack

LAYER	TECHNOLOGY	VERSION	ARCHITECTURAL ROLE & PURPOSE
Frontend Framework	React.js	19.2.7	Modern concurrent rendering, hooks, and component lifecycle for reactive UI.
Client Bundler	Vite	8.1.1	Lightning-fast HMR dev server and highly optimized production tree-shaking.
Design System	Tailwind CSS	4.3.3	Tailwind v4 zero-runtime CSS with modern dark UI palettes and micro-animations.
Client Routing	React Router DOM	7.18.1	Declarative multi-layout routing (Storefront Layout vs. Admin Layout).
Data Visualization	Recharts	3.10.0	Responsive sales charts, revenue trends, and top dishes analytics in POS.
Iconography	Lucide React	1.26.0	Consistent, lightweight SVG iconography across mobile and desktop.
HTTP Networking	Axios	1.18.1	Promise-based HTTP client with global loader triggers & response handling.
Backend Gateway	Express.js (Node.js)	5.2.1	High-performance REST API router, CORS configuration, and static serving.
Database & Cloud	Firebase Admin / Firestore	14.2.0	Real-time NoSQL document database storing items, orders, and customer logs.
File & Image Engine	Multer	2.2.0	Multipart form data parser for uploading food photography and thumbnails.
Date & Time Ops	Moment.js	2.30.1	Timezone-aware order timestamping, daily summary cuts, and filtering.
Deployment (Client)	Vercel	Production	Global Edge Network with Single Page App rewrites (chick-blast.vercel.app).

3. System Architecture & Data Pipeline

The system follows a decoupled Single Page Application (SPA) architecture with an Express.js API gateway and Google Cloud Firestore.

1. Storefront Client

React 19 SPA routes orders, cart state, and customer persistence via context providers.

2. API Gateway

Express 5 router executes validation, caching headers, and Multer multipart processing.

3. Firebase Cloud

Firestore NoSQL stores documents with atomic updates for live inventory and orders.

4. KDS & POS Admin

Live synchronization updates kitchen monitors and administrative dashboards.

4. Detailed Feature Specifications

Customer-Facing Storefront Features

- **Interactive Digital Catalog:** Real-time category selector tabs (*Crispy Fried Chicken, Burgers, Combos, Beverages, Fries & Sides, Desserts*) with immediate search matching.
- **Signature Hero Showcase:** Rotating dynamic carousel spotlighting signature recipes with spice ratings, prep times, and quick-add shortcuts.
- **Comprehensive Product Modal:** Shows detailed nutritional badges, portion sizes, spice level indicators, and certified FSSAI food hygiene tags.
- **Smart Multi-Channel Cart:**
 - Switch between **Takeaway, Dine-In, or Home Delivery**.
 - Subtotal, delivery surcharges, GST/tax calculation, and net billing automatically computed.
 - Instant customer profile recall: Name, phone number, and address remember past orders to provide frictionless one-click checkout.
- **Real-Time Visual Order Tracker:** Customers can enter their Order ID or Phone Number to monitor kitchen milestones:



- **Custom Food-Themed 404 Experience:** Playful fallback page ("Oops! This Chicken Flew Away 🐔") preventing lost conversions and guiding customers back to the menu.

Kitchen Display System (KDS) & Admin POS Features

- **Live Kitchen Stream (KDS):** Real-time card feed showing active tickets with elapsed timers, order itemizations, special requests, and single-click stage advances.
- **Executive Analytics Dashboard:** Key metrics at a glance: Total Daily/Weekly Gross Revenue, Completed Ticket Count, Average Ticket Size, and Recharts graphical performance curves.
- **Date-Filtered Order History:** Query orders across custom date spans (fromDate to toDate) with customer phone lookup and itemized audit receipts.
- **Catalog CRUD Management:** Add new menu items, edit prices, toggle stock availability, and update descriptions seamlessly.
- **Combo Builder Engine:** Configure combo deals grouping multiple items into bundled value meals with customized promotional discounts.
- **Asset Upload Pipeline:** Direct image uploading handled via Multer middleware into server storage with generated asset URLs.

5. REST API Specifications

METHOD	ENDPOINT	PARAMETERS / BODY	DESCRIPTION
GET	/api/health	None	Server status and health check probe.
GET	/api/items	?category=...	Fetch all active menu items and bundled combo meals.
POST	/api/items	JSON Item Payload	Add a new food item or combo package into catalog.
PUT	/api/items/:id	Updated Fields	Update pricing, stock availability, or description of an item.
DELETE	/api/items/:id	URL :id	Remove or deactivate an item from active menu listings.
GET	/api/orders	?fromDate=&toDate=&phone=	Retrieve orders list with flexible date and phone filtering.
GET	/api/orders/:id	URL :id	Fetch detailed progress status and receipt for a single order.
POST	/api/orders	Order Ticket Payload	Submit and store new customer order with line items and customer info.
PATCH	/api/orders/:id/status	{ "status": "Preparing" }	Advance order stage (Pending → Preparing → Packed → Delivered).
GET	/api/dashboard/stats	?range=today week month	Compute aggregate revenue, order counts, and top selling items.
GET	/api/customers	?phone=...	Query repeat customer information and historic order frequencies.
POST	/api/upload	Multipart Form File	Upload product imagery to server assets storage via Multer.

6. Core Database Schema (Firestore)

Order Schema (orders/{orderId})

```
{
  "orderId": "CB-84920",
  "customerName": "Arun Kumar",
  "customerPhone": "9876543210",
  "orderType": "Delivery", // Dine-In, Takeaway, Delivery
  "deliveryAddress": "Flat 4B, Emerald Apts, MG Road",
  "items": [
    {
      "id": "ck_01",
      "name": "6pc Hot Crispy Wings",
      "price": 349,
      "quantity": 2
    }
  ],
  "subtotal": 698,
  "tax": 34.9,
  "deliveryFee": 40,
  "total": 772.9,
  "status": "Preparing", // Pending, Preparing, Packed, Delivered
  "createdAt": "2026-09-11T14:20:00Z"
}
```

Item Schema (items/{itemId})

```
{
  "id": "ck_01",
  "name": "6pc Hot Crispy Wings",
  "category": "Crispy Chicken",
  "price": 349,
  "description": "Signature crispy wings coated in spicy pepper sauce",
  "image": "/uploads/wings_spicy.jpg",
  "isAvailable": true,
  "spiceLevel": "Extra Hot",
  "isCombo": false,
  "comboDetails": null,
  "rating": 4.9,
  "createdAt": "2026-09-01T10:00:00Z"
}
```

7. Codebase & Folder Organization

```
chick_blast/
├── public/                # Static branding assets, icons, and web manifest
├── server/                # Express.js REST API Server
│   ├── config/           # Firebase Admin credentials and Firestore instance
│   ├── screens/          # Domain-driven backend modular routes & controllers
│   │   ├── customers/    # Customer history models & lookup controller
│   │   ├── dashboard/    # Sales analytics aggregations and KPIs
│   │   ├── items/        # Menu item catalog & combo CRUD controllers
│   │   ├── orders/       # Order placement & status transition engine
│   │   └── upload/       # Multer multipart image upload router
│   └── index.js          # Express server entry point & middleware bindings
├── src/                  # React 19 Frontend Client
│   ├── admin/            # Kitchen POS & Management Interface
│   │   ├── components/   # Admin widgets, order tickets, modals
│   │   ├── layout/       # Sidebar navigation & admin header
│   │   └── pages/        # Dashboard, LiveOrders, OrderSummary, Items, ComboItems
│   ├── website/          # Customer-Facing Storefront
│   │   ├── components/   # ProductCard, DetailModal, QuantityControl
│   │   ├── layout/       # Public header with cart badges & footer
│   │   └── pages/        # Products (Menu), Cart, OrderStatus, NotFound (404)
│   ├── shared/           # Cross-application shared logic & UI tokens
│   │   ├── api/          # Axios endpoints & centralized API instances
│   │   ├── components/   # StatusPill, OrderBadge, ModernDatePicker, Loader
│   │   └── context/      # CartContext, CustomerContext (State Persistence)
│   ├── App.jsx           # React Router v7 routes declaration
│   └── main.jsx          # Application bootstrap & Context wrappers
├── vercel.json           # Vercel Single Page App routing configuration
├── package.json          # Scripts & dependency manifest
└── vite.config.js        # Vite build pipeline & local proxy settings
```

8. Getting Started & Deployment Guide

🌐 Live Production URL

The live customer application is accessible at: <https://chick-blast.vercel.app>

Local Environment Setup

```
# 1. Clone repository
git clone https://github.com/muraliofficial/chick_blast.git
cd chick_blast

# 2. Install dependencies
npm install

# 3. Configure environment variables in .env
PORT=3001
VITE_API_URL=http://localhost:3001
FIREBASE_PROJECT_ID=your_firebase_project_id
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
FIREBASE_CLIENT_EMAIL=your_client_email@your_project.iam.gserviceaccount.com

# 4. Start concurrent development server
npm run dev
```

🍷 Customer Storefront

Accessible at <http://localhost:5173>

👨‍🍳 Kitchen KDS & POS

Accessible at <http://localhost:5173/admin>

⚙️ Express REST API

Accessible at <http://localhost:3001/api>